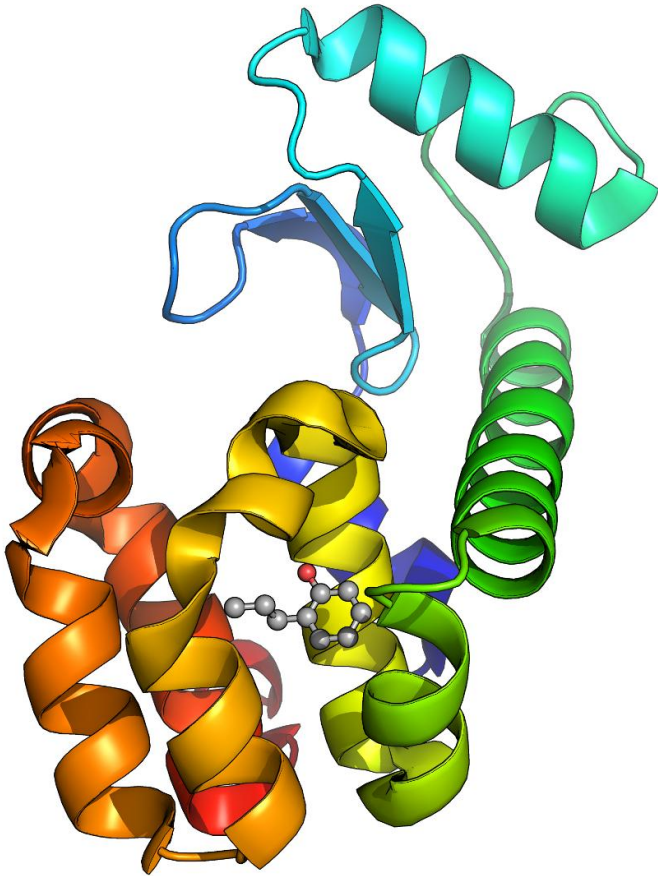


Justin A. Lemkul, Ph.D.
Virginia Tech Department of Biochemistry

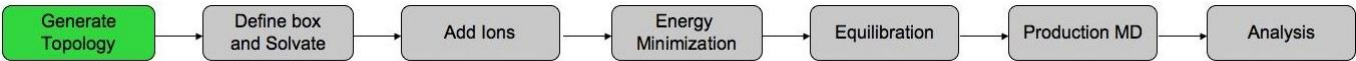


This example will guide a new user through the process of setting up a simulation system containing a protein (T4 lysozyme L99A/M102Q) in complex with a ligand. This tutorial focuses specifically on issues related to dealing with the ligand, assuming that the user is familiar with basic GROMACS operations and the contents of a topology. If this is not the case, please refer to [a more basic tutorial](#) before attempting this one.

This tutorial requires a GROMACS version in the 2025.x series.

GROMACS Tutorial

Step One: Prepare the Protein Topology



We must download the protein structure file we will be working with. For this tutorial, we will utilize T4 lysozyme L99A/M102Q (PDB code 3HTB). Go to the [RCSB](#) website and download the PDB text for the crystal structure.

Once you have downloaded the structure, you can visualize it using a viewing program such as VMD, Chimera, PyMOL, etc. Once you've had a look at the molecule, you are going to want to strip out the crystal waters, PO4, and BME. Note that such a procedure is not universally appropriate (i.e., the case of a bound active site water molecule). For our intentions here, we do not need crystal water or other species, which are just crystallization co-solvents. We will instead focus on the ligand called "JZ4," which is 2-propylphenol.

If you want a clean version of the .pdb file to check your work, you can download it [here](#). The problem we now face is that the JZ4 ligand is not a recognized entity in any of the force fields provided with GROMACS, so pdb2gmx will give a fatal error if you were try to pass this file through it. Topologies can only be assembled automatically if an entry for a building block is present in the .rtp (residue topology) file for the force field. Since this is not the case, we will prepare our system topology in two steps:

1. Prepare the protein topology with pdb2gmx
2. Prepare the ligand topology using external tools

Since we will be preparing these two topologies separately, we must save the protein and JZ4 ligand into separate coordinate files. Save the JZ4 coordinates like so:

```
grep JZ4 3HTB_clean.pdb > jz4.pdb
```

Then simply delete the JZ4 lines from 3HTB_clean.pdb. At this point, preparing the protein topology is trivial. The force field we will be using in this tutorial is CHARMM36, obtained from the [MacKerell lab website](#). While there, download the latest CHARMM36 force field tarball and the "cgenff_charmm2gmx.py" conversion script, which we will use later.

Unarchive the force field tarball in your working directory:

```
tar -zxvf charmm36-jul2022.ff.tgz
```

There should now be a "charmm36-jul2022.ff" subdirectory in your working directory. Write the topology for the T4 lysozyme with pdb2gmx:

```
gmx pdb2gmx -f 3HTB_clean.pdb -o 3HTB_processed.gro
```

Choose the default water model when prompted. The structure will be processed by pdb2gmx, and you will be prompted to choose a force field:

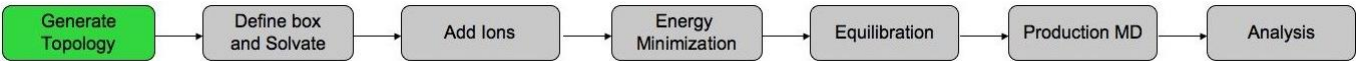
```
Select the Force Field:
From current directory:
1: CHARMM36 all-atom force field (July 2022)
```

```
From '/usr/local/gromacs/share/gromacs/top':
2: AMBER03 protein, nucleic AMBER94 (Duan et al., J. Comp. Chem. 24, 1999-2012, 2003)
3: AMBER94 force field (Cornell et al., JACS 117, 5179-5197, 1995)
4: AMBER96 protein, nucleic AMBER94 (Kollman et al., Acc. Chem. Res. 29, 461-469, 1996)
5: AMBER99 protein, nucleic AMBER94 (Wang et al., J. Comp. Chem. 21, 1049-1074, 2000)
6: AMBER99SB protein, nucleic AMBER94 (Hornak et al., Proteins 65, 712-725, 2006)
7: AMBER99SB-ILDN protein, nucleic AMBER94 (Lindorff-Larsen et al., Proteins 78, 1950-58, 2010)
8: AMBERGS force field (Garcia & Sanbonmatsu, PNAS 99, 2782-2787, 2002)
9: CHARMM27 all-atom force field (CHARM22 plus CMAP for proteins)
10: GROMOS96 43a1 force field
11: GROMOS96 43a2 force field (improved alkane dihedrals)
12: GROMOS96 45a3 force field (Schuler JCC 2001 22 1205)
13: GROMOS96 53a5 force field (JCC 2004 vol 25 pag 1656)
14: GROMOS96 53a6 force field (JCC 2004 vol 25 pag 1656)
15: GROMOS96 54a7 force field (Eur. Biophys. J. (2011), 40,, 843-856, DOI: 10.1007/s00249-011-0700-9)
16: OPLS-AA/L all-atom force field (2001 aminoacid dihedrals)
```

For this tutorial, choose the CHARMM36 force field (option 1), listed first under "From current directory" in the list.

GROMACS Tutorial

Step Two: Prepare the Ligand Topology



We must now deal with the ligand. But how does one come up with parameters for some species that the force field does not automatically recognize? Proper treatment of ligands is one of the most challenging tasks in molecular simulation. Force field authors spend years of their lives deriving self-consistent force fields, and it is no small task to introduce a new species into this framework. Force field parameters for any new species must be derived and validated in a manner that is consistent with the original force field.

For the OPLS, AMBER, and CHARMM force fields, this derivation often takes the form of various quantum mechanical calculations . The primary literature for these force fields describes the required procedure. For GROMOS force fields, parameterization methodology is less clear, relying on empirical fitting of condensed-phase behavior. That is, some initial charges and Lennard-Jones parameters are calculated for each atom type, evaluated for their accuracy, and refined. While the end result is very satisfactory, i.e. fluids resemble their real-world counterparts, the derivation process can be laborious and frustrating.

For this reason, automated tools are greatly preferred. For each force field, there are methodologies or software programs that purport to give parameters compatible with various force fields. Not all of them are equally accurate. For a few examples, refer to the following table:

AMBER	Antechamber acpype	Parametrizes molecules using GAFF A Python interface to Antechamber, writes GROMACS topologies
CHARMM	CGenFF	The official CHARMM General Force Field server
GROMOS96	ATB	A newer server for topology generation, uses GROMOS96 54A7
OPLS-AA	LigParGen	A server from the Jorgensen group to produce OPLS topologies

Add Hydrogen Atoms to JZ4

In this tutorial, we will be generating the JZ4 topology with the CGenFF server. Click the link in the table above to visit the site. Register for a (free) account and activate it. CGenFF requires a Sybyl .mol2 file as its input, to collect rudimentary atom type information and bonded connectivity. CHARMM is also an all-atom force field, meaning all H atoms are explicitly represented. Crystal structures do not typically assign H coordinates, so they have to be built in. To produce a .mol2 file and add H atoms, use the **Avogadro program**. Open jz4.pdb in Avogadro, and from the "Build" menu, choose "Add Hydrogens." Avogadro will build all of the H atoms onto the JZ4 ligand. Save a .mol2 file (File -> Save As... and choose Sybyl Mol2 from the drop-down menu) named "jz4.mol2."

Several corrections must be made to jz4.mol2 before it can be used. Open jz4.mol2 in your favorite plain-text editor and you will find:

```
@<TRIPOS>MOLECULE
****
 22 22 0 0 0
SMALL
GASTEIGER

@<TRIPOS>ATOM
 1 C4      24.2940 -24.1240 -0.0710 C.3  167 JZ4167 -0.0650
 2 C7      21.5530 -27.2140 -4.1120 C.ar  167 JZ4167 -0.0613
 3 C8      22.0680 -26.7470 -5.3310 C.ar  167 JZ4167 -0.0583
 4 C9      22.6710 -25.5120 -5.4480 C.ar  167 JZ4167 -0.0199
 5 C10     22.7690 -24.7300 -4.2950 C.ar  167 JZ4167  0.1200
 6 C11     21.6930 -26.4590 -2.9540 C.ar  167 JZ4167 -0.0551
 7 C12     22.2940 -25.1870 -3.0750 C.ar  167 JZ4167 -0.0060
 8 C13     22.4630 -24.4140 -1.8080 C.3   167 JZ4167 -0.0245
 9 C14     23.9250 -24.7040 -1.3940 C.3   167 JZ4167 -0.0518
10 OAB     23.4120 -23.5360 -4.3420 O.3   167 JZ4167 -0.5065
11 H       25.3133 -24.3619  0.1509 H     1 UNL1  0.0230
12 H       23.6591 -24.5327  0.6872 H     1 UNL1  0.0230
13 H       24.1744 -23.0611 -0.1016 H     1 UNL1  0.0230
14 H       21.0673 -28.1238 -4.0754 H     1 UNL1  0.0618
15 H       21.9931 -27.3472 -6.1672 H     1 UNL1  0.0619
16 H       23.0361 -25.1783 -6.3537 H     1 UNL1  0.0654
17 H       21.3701 -26.8143 -2.0405 H     1 UNL1  0.0621
18 H       21.7794 -24.7551 -1.0588 H     1 UNL1  0.0314
19 H       22.2659 -23.3694 -1.9301 H     1 UNL1  0.0314
20 H       24.5755 -24.2929 -2.1375 H     1 UNL1  0.0266
21 H       24.0241 -25.7662 -1.3110 H     1 UNL1  0.0266
22 H       23.7394 -23.2120 -5.1580 H     1 UNL1  0.2921

@<TRIPOS>BOND
 1 4 3 ar
 2 4 5 ar
 3 3 2 ar
```

```
4      10      5      1
5       5      7      ar
6       2      6      ar
7       7      6      ar
8       7      8      1
9       8      9      1
10      9      1      1
11      1     11      1
12      1     12      1
13      1     13      1
14      2     14      1
15      3     15      1
16      4     16      1
17      6     17      1
18      8     18      1
19      8     19      1
20      9     20      1
21      9     21      1
22     10     22      1
```

The first change that needs to be made is in the `MOLECULE` heading. Replace `*****` with `"JZ4,"` e.g.:

```
@<TRIPOS>MOLECULE
JZ4
```

Next, fix the residue names and numbers such that they are all the same. This `.mol2` file only contains one molecule, therefore there should only be one residue name and number specified. After editing, the corrected `ATOM` section of `jz4.mol2` should read:

```
@<TRIPOS>ATOM
1 C4      24.2940 -24.1240 -0.0710 C.3      1 JZ4      -0.0650
2 C7      21.5530 -27.2140 -4.1120 C.ar     1 JZ4      -0.0613
3 C8      22.0680 -26.7470 -5.3310 C.ar     1 JZ4      -0.0583
4 C9      22.6710 -25.5120 -5.4480 C.ar     1 JZ4      -0.0199
5 C10     22.7690 -24.7300 -4.2950 C.ar     1 JZ4       0.1200
6 C11     21.6930 -26.4590 -2.9540 C.ar     1 JZ4      -0.0551
7 C12     22.2940 -25.1870 -3.0750 C.ar     1 JZ4      -0.0060
8 C13     22.4630 -24.4140 -1.8080 C.3      1 JZ4      -0.0245
9 C14     23.9250 -24.7040 -1.3940 C.3      1 JZ4      -0.0518
10 OAB     23.4120 -23.5360 -4.3420 O.3      1 JZ4      -0.5065
11 H       25.3133 -24.3619  0.1509 H       1 JZ4       0.0230
12 H       23.6591 -24.5327  0.6872 H       1 JZ4       0.0230
13 H       24.1744 -23.0611 -0.1016 H       1 JZ4       0.0230
14 H       21.0673 -28.1238 -4.0754 H       1 JZ4       0.0618
15 H       21.9931 -27.3472 -6.1672 H       1 JZ4       0.0619
16 H       23.0361 -25.1783 -6.3537 H       1 JZ4       0.0654
17 H       21.3701 -26.8143 -2.0405 H       1 JZ4       0.0621
18 H       21.7794 -24.7551 -1.0588 H       1 JZ4       0.0314
19 H       22.2659 -23.3694 -1.9301 H       1 JZ4       0.0314
20 H       24.5755 -24.2929 -2.1375 H       1 JZ4       0.0266
21 H       24.0241 -25.7662 -1.3110 H       1 JZ4       0.0266
22 H       23.7394 -23.2120 -5.1580 H       1 JZ4       0.2921
```

Last, notice the strange bond order in the `@<TRIPOS>BOND` section. All programs seem to have their own method for generating this list, but not all are created equal. There will be issues in constructing a correct topology with matching coordinates if the bonds are not listed in ascending order. To fix this problem, download the [sort_mol2_bonds.pl](#) script I have written (change the `.txt` extension to `.pl`) and execute it:

```
perl sort_mol2_bonds.pl jz4.mol2 jz4_fix.mol2
```

Use `"jz4_fix.mol2"` in the next step.

Generate the JZ4 Topology with CGenFF

The `jz4_fix.mol2` file is now ready for use to produce a topology. Visit the CGenFF server, log into your account, and click "Upload molecule" at the top of the page. Upload `jz4_fix.mol2` and the CGenFF server will quickly return a topology in the form of a CHARMM "stream" file (extension `.str`). Save its contents from your web browser into a file called `"jz4.str."`

The CHARMM stream file contains all of the topology information - atom types, charges, and bonded connectivity. It also has sections for additional bonded parameters that were generated by analogy for any internal interactions not covered by the force field. CGenFF also provides penalty scores for each parameter, that is, an assessment of how reliable the assigned parameter is. Anything below 10 is considered acceptable for immediate use. Values from 10 - 50 imply that some validation of the topology is warranted, and any penalties larger than 50 generally require manual reparametrization. This penalty scoring is one of the most important features of the CGenFF server. Many other servers generate topologies and are "black boxes," which users are simply left to trust implicitly. Staking your entire research project on an automatic program without verification is very dangerous. A poor ligand topology can lead to significant wasted time and unreliable results. Always validate the topologies of newly parametrized species! At minimum, check the magnitudes of charges and the atom types assigned to the ligand against existing moieties in the force field.

Examine the contents of `jz4.str` and look at the penalties for the charges and the new dihedral parameters. All of them are very low, suggesting that this topology is of very good quality and can be used directly for our simulation. Download the JZ4 topology in GROMACS format, which is provided by the CGenFF server (click "Convert to GROMACS" and then "Download GROMACS format" to obtain a ZIP archive with the coordinates and topology).

The files produced by the CGenFF server will require some modification to be used directly. The server will provide a topology called `"jz4_fix_gmx.top,"` which is a GROMACS system topology, meaning it is intended for standalone use. We will need to adapt it for our purpose. First, copy it to a file called `"jz4_gmx.itp"`:

```
cp jz4_fix_gmx.top jz4_gmx.itp
```

Make the following modifications to `"jz4_gmx.itp"`:

1. Remove the `#include "charmm36.ff/forcefield.itp"` line
2. Replace the `#include "charmm36.ff/jz4_ffbonded.itp"` line with the actual content of that file
3. Remove all lines from `"`; Include water topology" to the end of the file
4. Rename the `[moleculetype]` from `Other` to `JZ4`
5. Change `#include "posre.itp"` to `#include "posre_jz4.itp"`

A summary of the changes made in `"jz4_gmx.itp"` are shown below, with added/modified lines in **blue** and deleted lines shown in **red**.

```
; Topology generated using molcal v1.0
; For use with CGenFF force-field version 4.6
;
```

```
; Include forcefield parameters
#include "../charmm36.ff/forcefield.itp"
;
; Include ligand specific parameters
# include "../charmm36.ff/jz4_ffbonded.itp"
[ dihedraltypes ]
;      i      j      k      l  func      phi0      kphi  mult
   CG2R61    CG321    CG321    CG331    9      0.000000    0.167360    3
;
[ moleculetype ]
; Name          nrexcl
JZ4              3
;
[ atoms ]
; Include Position restraint file
#ifdef POSRES
#include "posre_jz4.itp"
#endif

; Include water topology
#include "../charmm36.ff/tip3p.itp"

#ifdef POSRES_WATER
; Position restraint for each water oxygen
[ position_restraints ]
;  i  funct      fcx      fcy      fcz
   1   1      1000      1000      1000
#endif

; Include topology for ions
#include "../charmm36.ff/ions.itp"

[ system ]
; Name
   Ligand

[ molecules ]
; Compound      #mols
Other              1
```

Build the Complex

From pdb2gmx, we have a file called "3HTB_processed.gro" that contains the processed, force field -compliant structure of our protein. We also have "jz4_fix_gmx.pdb" from the CGENFF server that has all of the necessary H atoms and matches the atom names in the JZ4 topology. Convert this .pdb file to .gro format with editconf:

```
gmx editconf -f jz4_ini.pdb -o jz4.gro
```

Copy 3HTB_processed.gro to a new file to be manipulated, for instance, call it "complex.gro," as the addition of JZ4 to the p protein will generate our protein-ligand complex. Next, simply copy the coordinate section of jz4.gro and paste it into complex.gro, below the last line of the protein atoms, and before the box vectors, like so:

```
163ASN      C 1691      0.621      -0.740      -0.126
163ASN      O1 1692      0.624      -0.616      -0.140
163ASN      O2 1693      0.683      -0.703      -0.011
  5.99500    5.19182    9.66100      0.00000      0.00000     -2.99750      0.00000      0.00000      0.00000
```

becomes (added text in **bold green**)...

```
163ASN      C 1691      0.621      -0.740      -0.126
163ASN      O1 1692      0.624      -0.616      -0.140
163ASN      O2 1693      0.683      -0.703      -0.011
  1jz4      C1   1      2.429      -2.412      -0.007
  1jz4      C2   2      2.155      -2.721      -0.411
  1jz4      C3   3      2.207      -2.675      -0.533
  1jz4      C4   4      2.267      -2.551      -0.545
  1jz4      C5   5      2.277      -2.473      -0.430
  1jz4      C6   6      2.169      -2.646      -0.295
  1jz4      C7   7      2.229      -2.519      -0.308
  1jz4      C8   8      2.246      -2.441      -0.181
  1jz4      C9   9      2.392      -2.470      -0.139
  1jz4      O10  10     2.341      -2.354      -0.434
  1jz4      H11  11     2.531      -2.436       0.015
  1jz4      H12  12     2.366      -2.453       0.069
  1jz4      H13  13     2.417      -2.306      -0.010
  1jz4      H14  14     2.107      -2.812      -0.407
  1jz4      H15  15     2.199      -2.735      -0.617
  1jz4      H16  16     2.304      -2.518      -0.635
  1jz4      H17  17     2.137      -2.681      -0.204
  1jz4      H18  18     2.178      -2.476      -0.106
  1jz4      H19  19     2.227      -2.337      -0.193
  1jz4      H20  20     2.458      -2.429      -0.214
  1jz4      H21  21     2.402      -2.577      -0.131
  1jz4      H22  22     2.374      -2.321      -0.516
  5.99500    5.19182    9.66100      0.00000      0.00000     -2.99750      0.00000      0.00000      0.00000
```

Since we have added 22 more atoms into the .gro file, increment the second line of complex.gro to reflect this change. There should be 2636 atoms in the coordinate file now.

Build the Topology

Including the parameters for the JZ4 ligand in the system topology is very easy. The ligand introduces new dihedral parameters, which means it is essential that we include the ligand topology at the **TOP** of topol.top, thus insert an #include statement to add these parameters:

```
; Include forcefield parameters
#include "../charmm36-jul2022.ff/forcefield.itp"

; Include ligand parameters
#include "jz4_gmx.itp"

[ moleculetype ]
; Name          nrexcl
```


The placement of this `#include` statement is critical - it must appear before any `[ moleculetype ]` entry because all parameters have to be defined before any molecules can be constructed. It must also appear **AFTER** the `#include` statement for the parent force field, because all atom types have to be known before bonded parameters can be introduced that make use of them.

The last adjustment to be made is in the `[ molecules ]` directive. To account for the fact that there is a new molecule in `complex.gro`, we have to add it here, like so:

```
[ molecules ]
; Compound      #mols
Protein_chain_A    1
JZ4                1
```

The topology and coordinate file are now in agreement with respect to the contents of the system.

GROMACS Tutorial

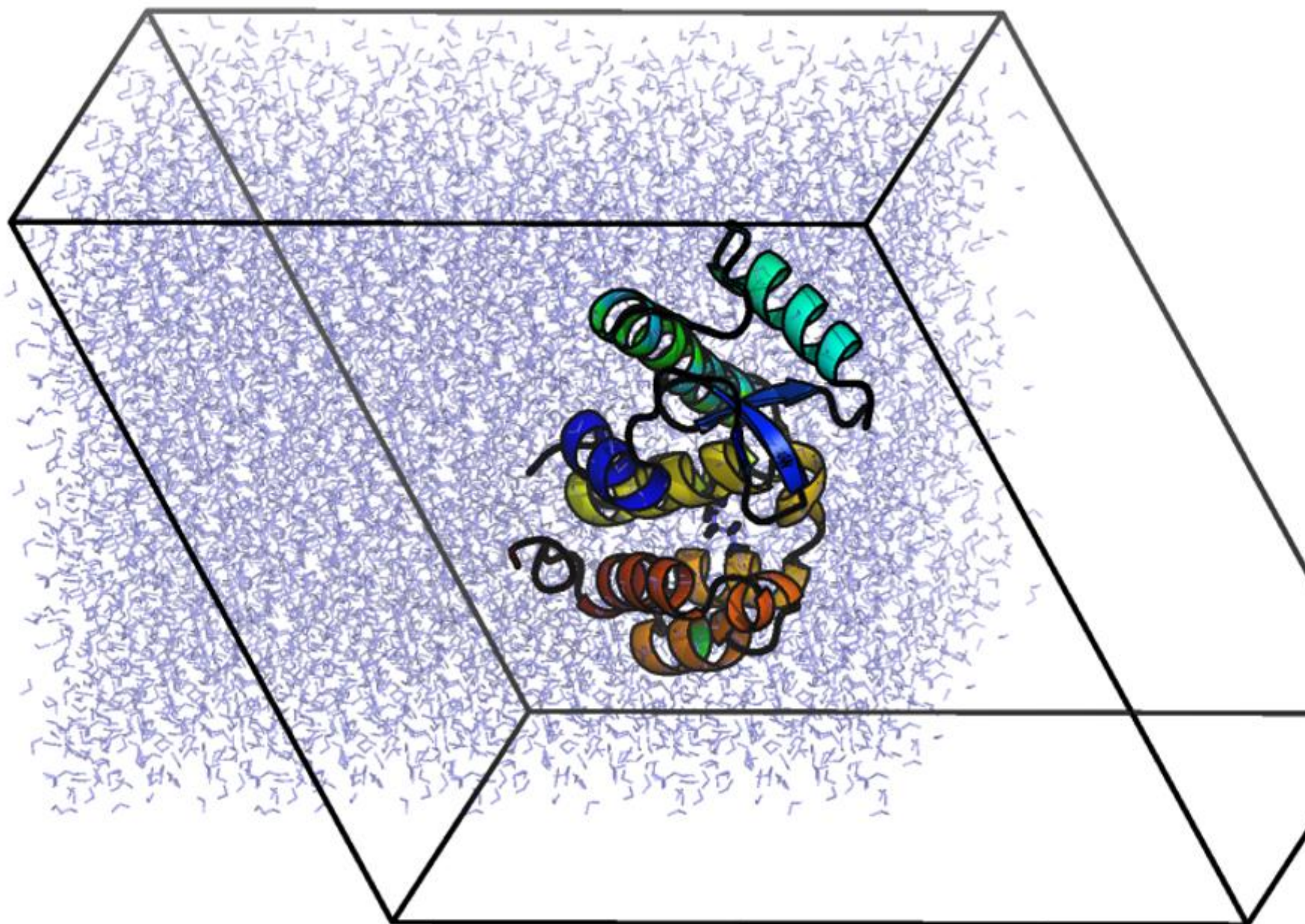
Step Three: Defining the Unit Cell & Adding Solvent



At this point, the workflow is just like any other MD simulation. We will define the unit cell and fill it with water.

```
gmx editconf -f complex.gro -o newbox.gro -bt dodecahedron -d 1.0
gmx solvate -cp newbox.gro -cs spc216.gro -p topol.top -o solv.gro
```

Upon visualizing `solv.gro`, you may wonder why `editconf` did not produce the requested dodecahedral unit cell shape, as the system will look something like this:



GROMACS programs always use the most numerically efficient representation of the coordinates, one that has everything re-wrapped into a triclinic unit cell. The physical calculations that `mdrun` performs can be carried out equivalently with different coordinate wrapping, so the most efficient is preferred. The desired unit cell shape can be recovered later, following the generation of a `.tpr` file.

Step Four: Adding Ions



We now have a solvated system that contains a charged protein. The output of `pdb2gmx` told us that the protein has a net charge of `+6e` (based on its amino acid composition). If you missed this information in the `pdb2gmx` output, look at the last line of your `[ atoms ]` directive in `topol.top`; it should read (in part) "qtot 6." Since life does not exist at a net charge, we must add ions to our system.

Use `grompp` to assemble a `.tpr` file, using any `.mdp` file. I use an `.mdp` file for running energy minimization, since they require the fewest parameters and are thus the easiest to maintain. For example, [this one](#).

```
gmx grompp -f ions.mdp -c solv.gro -p topol.top -o ions.tpr
```

We now pass our `.tpr` file to `genion`:

```
gmx genion -s ions.tpr -o solv_ions.gro -p topol.top -pname NA -nname CL -neutral
```

The names of the ions specified with `-pname` and `-nname` were force field-specific in previous versions of GROMACS, but were standardized in version 4.5. The specified atom names are always the elemental symbol in all capital letters, along with the `[ moleculetype ]`. Residue names may or may not append the sign of the charge (+/-). Refer to `ions.itp` for proper nomenclature if you encounter difficulties.

Your `[ molecules ]` directive should now look like:

```
[ molecules ]
; Compound      #mols
Protein_chain_A    1
JZ4                1
SOL               10228
CL                 6
```

GROMACS Tutorial

Step Five: Energy Minimization



Now that the system is assembled, create the binary input using `grompp` using [this](#) input parameter file:

```
gmx grompp -f em.mdp -c solv_ions.gro -p topol.top -o em.tpr
```

Make sure you have been updating your `topol.top` file when running `genbox` and `genion`, or else you will get lots of nasty error messages ("number of coordinates in coordinate file does not match topology," etc).

We are now ready to invoke `mdrun` to carry out the EM:

```
gmx mdrun -v -deffnm em
```

For me, the system converged relatively quickly:

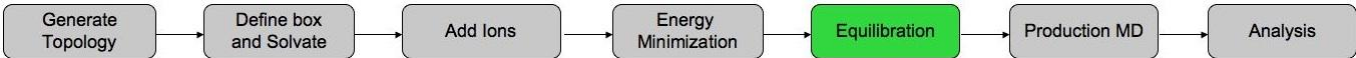
```
Steepest Descents converged to Fmax < 1000 in 134 steps
Potential Energy = -4.8970009e+05
Maximum force    = 8.2663605e+02 on atom 27
Norm of force    = 5.7744695e+01
```

As in the [lysozyme tutorial](#), it is possible to monitor various components of the potential energy using the energy module. I will not illustrate these principles here. Have fun.

Now that our system is at an energy minimum, we can begin real dynamics.

GROMACS Tutorial

Step Six: Equilibration



Equilibrating our protein-ligand complex will be much like equilibrating any other system containing a protein in water. There are a few special considerations, in this case:

- 1. Applying restraints to the ligand
- 2. Treatment of temperature coupling groups

Restraining the Ligand

To restrain the ligand, we will need to generate a position restraint topology for it. First, create an index group for JZ4 that contains only its non-hydrogen atoms:

```
gmx make_ndx -f jz4.gro -o index_jz4.ndx
...
> 0 & ! a H*
> q
```

Then, execute the genrestr module and select this newly created index group (which will be group 3 in the index_jz4.ndx file):

```
gmx genrestr -f jz4.gro -n index_jz4.ndx -o posre_jz4.itp -fc 1000 1000 1000
```

The previous edits to the ligand topology prepared us for this step, such that we now have the "posre_jz4.itp" file that allows us to restrain the ligand when position restraints are invoked.

If you want a bit more control during equilibration, i.e. restraining the protein and ligand independently, you could instead control the inclusion of the ligand position restraint file in a different #ifdef block, like so:

```
; Ligand position restraints
#ifdef POSRES_LIG
#include "posre_jz4.itp"
#endif
```

In the latter case, to restrain **both** the protein and the ligand, we would need to specify `define = -DPOSRES -DPOSRES_LIG` in the .mdp file. How you want to treat your system is up to you. These examples are meant only to illustrate the flexibility GROMACS provides. For a standard equilibration procedure, restraining the protein and ligand simultaneously is probably sufficient. Your own needs may vary.

Thermostats

Proper control of temperature coupling is a sensitive issue. Coupling every moleculetype to its own thermostating group is a bad idea, though it may seem like this approach may be more accurate. For instance, if you do the following:

```
tc-grps = Protein JZ4 SOL CL
```

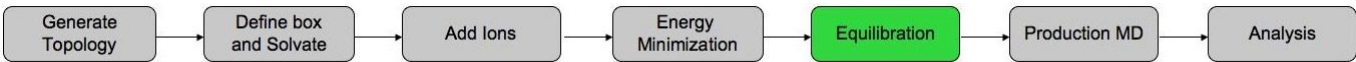
your system will probably blow up, since the temperature coupling algorithms are not stable enough to control the fluctuations in kinetic energy that groups with a few atoms (i.e., JZ4 and CL) will produce. **Do not couple every single species in your system separately.**

Proceed with *NVT* equilibration using **this** .mdp file.

```
gmx grompp -f nvt.mdp -c em.gro -r em.gro -p topol.top -o nvt.tpr
gmx mdrun -deffnm nvt
```

GROMACS Tutorial

Step Seven: Equilibration, Part 2



Once the *NVT* simulation is complete, proceed to *NPT* with **this** .mdp file:

```
gmx grompp -f npt.mdp -c nvt.gro -t nvt.cpt -r nvt.gro -p topol.top -o npt.tpr
gmx mdrun -deffnm npt
```

GROMACS Tutorial

Step Eight: Production MD



Upon completion of the two equilibration phases, the system is now well-equilibrated at the desired temperature and pressure. We are now ready to release the position restraints and run production MD for data collection. The process is just like we have seen before, as we will make use of the checkpoint file (which in this case now contains preserve pressure coupling information) to grompp. We will run a 10-ns MD simulation, the script for which can be found **here**.

```
gmx grompp -f md.mdp -c npt.gro -t npt.cpt -p topol.top -o md_0_10.tpr
gmx mdrun -deffnm md_0_10
```

GROMACS Tutorial

Step Nine: Analysis



Recentering and Rewrapping Coordinates

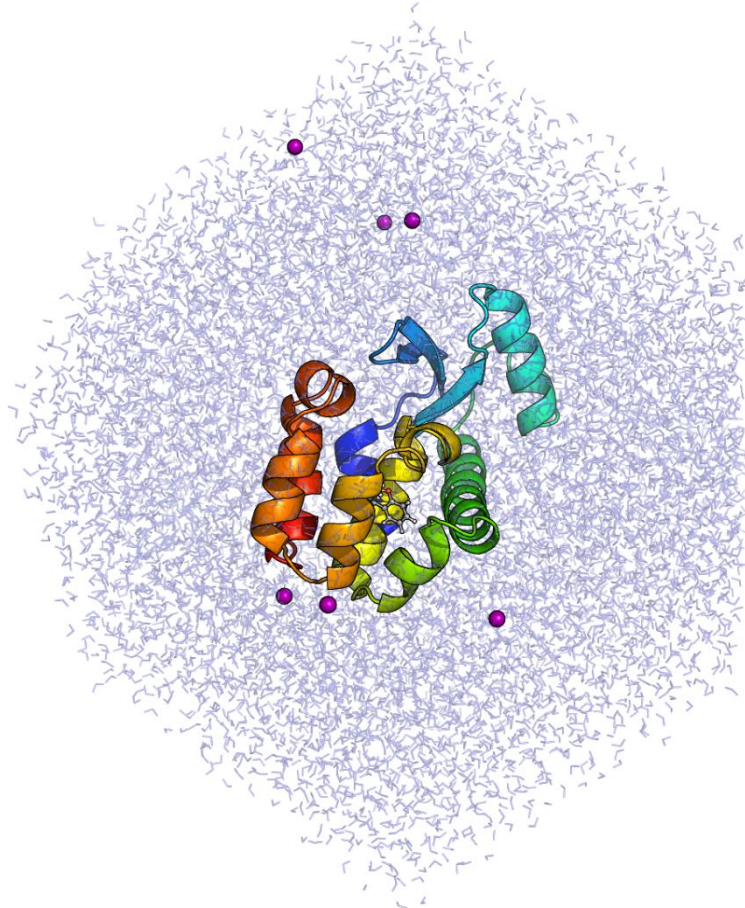
As in any simulation conducted with periodic boundary conditions, molecules may appear "broken" or may "jump" back and forth across the box. To re-center the protein and re-wrap the molecules within the unit cell to recover the desired rhombic dodecahedral shape, invoke trjconv:

```
gmx trjconv -s md_0_10.tpr -f md_0_10.xtc -o md_0_10_center.xtc -center -pbc mol -ur compact
```


Choose "Protein" for centering and "System" for output. Note that centering complexes (protein-ligand, protein-protein) may be difficult for longer simulations involving many jumps across periodic boundaries. In those instances (particularly in protein-protein complexes), it may be necessary to create a custom index group to use for centering, corresponding to the active site of one protein or the interfacial residues of one monomer in a complex.

To extract the first frame (t = 0 ns) of the trajectory, use `trjconv -dump` with the re-centered trajectory:

```
gmx trjconv -s md_0_10.tpr -f md_0_10_center.xtc -o start.pdb -dump 0
```



For even smoother visualization, it may be beneficial to perform rotational and translational fitting. Execute `trjconv` as follows:

```
gmx trjconv -s md_0_10.tpr -f md_0_10_center.xtc -o md_0_10_fit.xtc -fit rot+trans
```

Choose "Backbone" to perform least-squares fitting to the protein backbone, and "System" for output. Note that simultaneous PBC re-wrapping and fitting of the coordinates is mathematically incompatible. Should you wish to perform fitting (which is useful for visualization, but not necessary for most analysis routines), carry out these coordinate manipulations separately, as indicated here.

Analyzing Protein-Ligand Interactions and Ligand Dynamics

This tutorial cannot possibly cover all analysis methods that you may wish to perform. A few basic operations will be illustrated here.

The 2-propylphenol ligand can engage in a hydrogen bond with the Gln102 side chain. The GROMACS `hbond` module can easily be employed to calculate the number of hydrogen bonds between any groups of atoms, but in our case, the only values will be 1 or 0. For a more detailed look at how the ligand is interacting with Gln102, we will compute the distance between the hydroxyl group of JZ4 and the carbonyl O atom of Gln102. For a hydrogen bond to be formed, the typical criterion is that the donor and acceptor atoms will be separated by a distance ≤ 0.36 nm (3.5 Å). Use the distance module to calculate the distance over the course of the trajectory, using command-line selection syntax (see `gmx help selections` for examples and more syntax).

```
gmx distance -s md_0_10.tpr -f md_0_10_center.xtc -select 'resname "jz4" and name O10 plus resid 102 and name OE1' -oall
```

The average distance is 0.43 ± 0.17 nm (4.3 ± 1.7 Å), which is slightly longer than the typical threshold for a hydrogen bond. Over the last 5 ns of the simulation (which can be calculated by adding `-b 5000` to the command above), the average distance is 0.30 ± 0.03 nm (3.0 ± 0.3 Å), consistent with a hydrogen bond. Therefore, after some initial equilibration time, the ligand settles into a hydrogen-bonded pose, at least with respect to the distance between the two oxygen atoms.

The second criterion usually applied in determining the presence of a hydrogen bond is the angle formed among the donor, hydrogen, and acceptor atoms. There are different conventions for calculating the angle. In the GROMACS `hbond` module, the angle is defined as hydrogen-donor-acceptor, and this angle should be $\leq 30^\circ$. To perform this analysis, first create index groups for the donor atoms (which must include both the donor heavy atom and the bonded hydrogen) and the acceptor atom:

```
gmx make_ndx -f em.gro
...
> 13 & a O10 | a H22
(creates group 20)
> 1 & r 102 & a OE1
(creates group 21)
> 20 | 21
> q
```

Analyze the angle formed by these three atoms using the `angle` module:

```
gmx angle -f md_0_10_center.xtc -n index.ndx -ov angle.xvg
```

Choose group 22 (the group we just created) when prompted. Note that `angle` takes no `-s` argument, unlike most other GROMACS analysis modules. Angle calculations do not require mass or periodicity information, atom names, etc. so the trajectory is processed without a `.tpr` or coordinate file. The value returned by the command should be roughly $32 \pm 15^\circ$ ($24 \pm 7^\circ$ over the last 5 ns). This outcome may be somewhat unexpected, as the index group we constructed was in the order of O10, H22, OE1, which would correspond to the donor-hydrogen-acceptor distance, which we expect to be close to linear ($\sim 180^\circ$). Inspect the contents of the index file and you will find:

```
[ jz4_&_O10_H22_Protein_&_r_102_&_OE1 ]
1616 2624 2636
```


make_ndx has sorted the atom numbers automatically from low to high, thus the outcome of the calculation is the acceptor-donor-hydrogen angle, the same angle that the hbond module would have calculated. So the result is consistent with formation of a hydrogen bond, since it is $\leq 30^\circ$. To get the desired angle of donor-hydrogen-acceptor, we would have to manually edit the index group in a text file to reorder the atom numbers (2624 2636 1616). Re-running the angle calculation with this index group yields an average value of $139 \pm 15^\circ$ ($146 \pm 9^\circ$ over the last 5 ns).

Finally, we may be interested in quantifying how much the ligand binding pose has changed over the course of the simulation. To compute a heavy-atom RMSD of just JZ4, create a new index group for it:

```
gmx make_ndx -f em.gro -n index.ndx
...
> 13 & ! a H*
> name 23 JZ4_Heavy
> q
```

Execute the rms module, choosing "Backbone" for least-squares fitting and "JZ4_Heavy" for the RMSD calculation. By doing so, the overall rotation and translation of the protein is removed via fitting and the RMSD reported is how much the JZ4 position has varied relative to the protein, which is a good indicator of how well the binding pose was preserved during the simulation.

```
gmx rms -s em.tpr -f md_0_10_center.xtc -n index.ndx -tu ns -o rmsd_jz4.xvg
```

The calculated RMSD should be about 0.15 ± 0.06 nm (1.5 ± 0.6 Å), indicating only a very small change in the ligand's position.

Protein-Ligand Interaction Energy

To quantify the strength of the interaction between JZ4 and T4 lysozyme, it may be useful to compute the nonbonded interaction energy between these two species. GROMACS has the ability to decompose the short-range nonbonded energies between any number of defined groups. It is important to note that this quantity is **NOT** a free energy or a binding energy. In fact, most force fields are not parametrized in such a way that this quantity is actually physically meaningful. CHARMM is parametrized to specifically target quantum mechanical interaction energies with water, so it is intrinsically balanced against meaningful quantities, and as such the interaction energy can be useful.

Calculation of an interaction energy is carried out via the `energygrps` keyword in the .mdp file. Despite being an .mdp keyword, interaction energy calculations should not be considered part of a normal simulation. Decomposition of the short-range energies is incompatible with running on a GPU and also slows the calculation down unnecessarily. The mdrun module does not need to do this extra work to perform a valid simulation. As such, only compute interaction energies as a part of your analysis, not your dynamics. Create a new .tpr file from an .mdp file that has `energygrps = Protein JZ4` defined, like **this one**:

```
gmx grompp -f ie.mdp -c npt.gro -t npt.cpt -p topol.top -n index.ndx -o ie.tpr
```

Next, invoke mdrun with the -rerun option to recalculate energies from the existing simulation trajectory:

```
gmx mdrun -deffnm ie -rerun md_0_10.xtc -nb cpu
```

Note the use of -deffnm to read ie.tpr and write all output files to ie.* as their file names. The -rerun option takes the name of the trajectory for which you want to recompute energies, and -nb cpu tells mdrun to only attempt to run on CPU hardware and ignore any GPU that might be available. As stated above, this type of calculation cannot be performed on a GPU. The rerun should be very fast, completing in just a few minutes.

Extract the energy terms of interest via the energy module. The terms we are interested in are Coul-SR:Protein-jz4 and LJ-SR:Protein-jz4.

```
gmx energy -f ie.edr -o interaction_energy.xvg
```

The average short-range Coulombic interaction energy is -16.5 ± 10.0 kJ mol⁻¹ and the short-range Lennard-Jones energy is -96.8 ± 7.4 kJ mol⁻¹. It may be tempting to draw conclusions from the relative magnitudes of these quantities, but even though CHARMM was parametrized against explicit water interaction energies, decomposition of interaction energies further into these components is not necessarily real. There is no way to experimentally verify these quantities, so it is impossible to know whether they are meaningful. The **total** interaction energy, however, is useful in this case and can be viewed as an approximation of the enthalpy between the two species. This value (after propagating the error according to the standard formula for addition of two quantities) is -113.3 ± 12.4 kJ mol⁻¹.

GROMACS Tutorial

Summary



You have now conducted a molecular dynamics simulation of a protein-ligand complex with GROMACS. This tutorial should not be viewed as comprehensive. There are many more types of simulations that one can conduct with GROMACS (free energy calculations, non-equilibrium MD, and normal modes analysis, just to name a few). You should also review the literature and the GROMACS manual for adjustments to the .mdp files provided here for efficiency and accuracy purposes.

If you have suggestions for improving this tutorial, if you notice a mistake, or if anything is otherwise unclear, please feel free to **email me**. Please note: this is not an invitation to email me for GROMACS problems. I do not advertise myself as a private tutor or personal help service. That's what the **gmx-users list** is for. I may help you there, but only in the context of providing service to the community as a whole, not just the end user.

Happy simulating!